EliosBase — Remaining Development Tasks

Date: March 18, 2026 **Status:** Frontend + Database infrastructure complete. Items below need implementation.

Dev 1 — On-Chain Integration & Wallet

1. Live Wallet Stats

File: `frontend/src/app/app/wallet/page.tsx` **Current:** Balance, In Escrow, Total Earned, Staked are hardcoded values. **Required:**

- Read ETH balance from Base network using viem's `getBalance`
- Calculate "In Escrow" by summing active escrow_lock transactions minus released
- Calculate "Total Earned" from reward transactions in the DB
- Read staked amount from staking contract (or DB if no contract yet)
- Display real values instead of static strings

2. On-Chain Transaction Syncing

File: `frontend/src/app/api/transactions/sync/route.ts` **Current:** API route exists but nothing calls it. Transaction history shows seed data only. **Required:**

- Sync on-chain transactions from Base network to the `transactions` table
- Index by user wallet address
- Poll or use websocket for new transactions
- Update transaction status (pending → confirmed) based on on-chain receipts

3. Hire Agent Flow

Files: `frontend/src/components/dashboard/AgentCard.tsx`, new API route **Current:** "Hire" button toggles visually but has no backend logic. **Required:**

- POST to a new `/api/agents/[id]/hire` endpoint
- Create an escrow transaction (lock funds)
- Assign the hired agent to the user's task
- Create activity event for the hire action
- Update agent status to "busy" if appropriate

4. Live Dashboard Stats

File: `frontend/src/app/app/page.tsx`, `frontend/src/lib/constants.ts` **Current:** Active Agents (2,847), Tasks in Progress (1,204), TVL ($14.2M), ZK Proofs (8,491) are static. **Required:**

- Query `agents` table: COUNT where status != 'offline'
- Query `tasks` table: COUNT where status = 'active'
- TVL: sum of all escrow_lock amounts minus escrow_release amounts (or read from contract)
- ZK Proofs: COUNT of tasks where zk_proof_id IS NOT NULL
- Create a new API route `/api/stats` that returns computed values
- Replace static constants with live data hook

Dev 2 — Security, Alerts & Automation

5. Security Alert Generation

Files: `frontend/src/app/api/security/alerts/route.ts`, new service logic **Current:** Shows seed data only. No system generates new alerts. **Required:**

- Define alert triggers (e.g., agent exceeds spending limit, unusual transaction pattern, proof verification timeout)
- Create alerts programmatically when triggers fire
- Add POST endpoint or service-role insert for creating alerts
- Mark alerts as resolved via PATCH endpoint
- Add "Resolve" button to SecurityAlert component in the UI

6. Guardrail Enforcement

Files: `frontend/src/app/api/security/guardrails/route.ts`, task/agent API routes **Current:** Guardrails are static DB entries with no enforcement logic. **Required:**

- Spending Limits: check task reward against per-task cap before allowing submission
- Rate Limiting: track API calls per agent, deny if over threshold
- Memory Isolation: enforce in agent execution layer
- Increment `triggered_count` when a guardrail fires
- Update guardrail status to 'triggered' when appropriate
- Create security alert when guardrail triggers
- Add PATCH endpoint for toggling guardrail status (active/paused)

7. Live Security Stats

File: `frontend/src/app/app/security/page.tsx` **Current:** Threats Blocked (247), Guardrails Active (5/6), Proofs Verified (8,491), Uptime (99.97%) are hardcoded. **Required:**

- Threats Blocked: COUNT of security_alerts
- Guardrails Active: COUNT where status = 'active' / total COUNT
- Proofs Verified: COUNT of tasks with zk_proof_id IS NOT NULL
- Uptime: track via external monitoring or compute from audit_log
- Create `/api/security/stats` endpoint
- Replace static values with live data

8. Audit Log Auto-Population

Files: API routes across the app **Current:** Audit log shows seed data. Only task submission creates activity events. **Required:**

- Log all significant actions to `audit_log` table:
  - TASK_CREATE, TASK_ASSIGN, TASK_COMPLETE
  - ESCROW_LOCK, ESCROW_RELEASE
  - AGENT_REGISTER, AGENT_HIRE
  - SPENDING_LIMIT violations
  - PROOF_SUBMIT, PROOF_VERIFY
- Add audit logging helper function used across API routes
- Include actor (wallet address or agent ID) and target in each entry

9. Activity Feed Auto-Population

Files: All API routes that modify data **Current:** Only task submission creates an activity event. **Required:**

- Generate activity events for:
  - Task status changes (assigned, executing, completed)
  - Payment/escrow events
  - Agent milestones (tasks completed thresholds)
  - Security events (guardrail triggered, alert created)
  - Proof verifications
- Use consistent message format matching existing seed data style

---

Reference

Database Tables

All 8 tables are created and seeded: `users`, `agents`, `tasks`, `transactions`, `security_alerts`, `guardrails`, `audit_log`, `activity_events`

Existing API Routes

Route	Methods	Auth
`/api/auth/nonce`	GET	No
`/api/auth/verify`	POST	No
`/api/auth/session`	GET	No
`/api/auth/logout`	POST	No
`/api/agents`	GET	No
`/api/agents/[id]`	GET	No
`/api/agents/register`	POST	Yes
`/api/tasks`	GET, POST	POST: Yes
`/api/tasks/[id]`	GET, PATCH	PATCH: Yes
`/api/transactions`	GET	Yes
`/api/transactions/sync`	POST	Yes
`/api/security/alerts`	GET	Yes
`/api/security/guardrails`	GET	Yes
`/api/security/audit-log`	GET	Yes
`/api/activity`	GET	No

Tech Stack

- Next.js 16.1.7, React 19, TypeScript
- Supabase (PostgreSQL + Realtime)
- wagmi + viem (Base network, chain ID 8453)
- iron-session (SIWE auth)

- @tanstack/react-query
- Tailwind CSS 4

Key Files

- Types: `src/lib/types/index.ts`
- Transforms: `src/lib/transforms.ts`
- Auth: `src/providers/AuthProvider.tsx`, `src/hooks/useSiwe.ts`
- Supabase clients: `src/lib/supabase/client.ts` (browser), `src/lib/supabase/server.ts` (service role)
- Session: `src/lib/session.ts`